

Capítulo 11

Redes Privadas

Al terminar este capítulo, entenderás:

- Qué es una red privada y por qué es útil.
- Los diferentes tipos de redes privadas: LAN, interredes privadas, redes híbridas, intranets y extranets.
- Qué es el direccionamiento privado y cuáles son bloques de direcciones reservadas.
- Qué el NAT, qué problema resuelve y cuáles provoca.

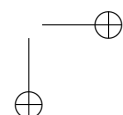
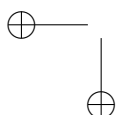
Una red privada, es solo eso, una red¹ de uso privado, en contraposición a una red pública, que es utilizada por el tráfico de mucha gente. Su objetivo principal suele ser compartir recursos dentro de una organización, por lo que todos los elementos físicos que conforman la red (computadores, dispositivos de interconexión, cableado, etc.) están bajo el control exclusivo de esa organización, y por tanto, también es responsable de su diseño, gestión y explotación.

El concepto «red privada» no es equivalente a «red aislada». La red privada más común hoy día es la red WiFi doméstica que encontramos en la mayoría de los hogares y que, obviamente, está conectada a Internet a través del llamado «router doméstico» y la red del ISP.

11.1. Líneas alquiladas

También una interred puede ser privada, es decir, puede ser una colección de LANs interconectadas mediante routers, ya sea en una o varias localizaciones, todo eso bajo el control y uso exclusivo de una misma organización. Durante los años 80 a 90, cuando Internet no existía o el acceso era demasiado caro, era habitual que muchas empresas tuvieran una red privada que conectaba las LAN de sus oficinas —por ejemplo, los concesionarios de una

¹Habitualmente una LAN



marca de automóviles. Para conectar estas oficinas entre sí, la organización podía instalar cableado propio o bien alquilar líneas a una compañía de telecomunicaciones². La primera opción solo era económicamente viable para distancias muy cortas (muy pocos cientos de metros). Este planteamiento es el que muestra la Figura 11.1.

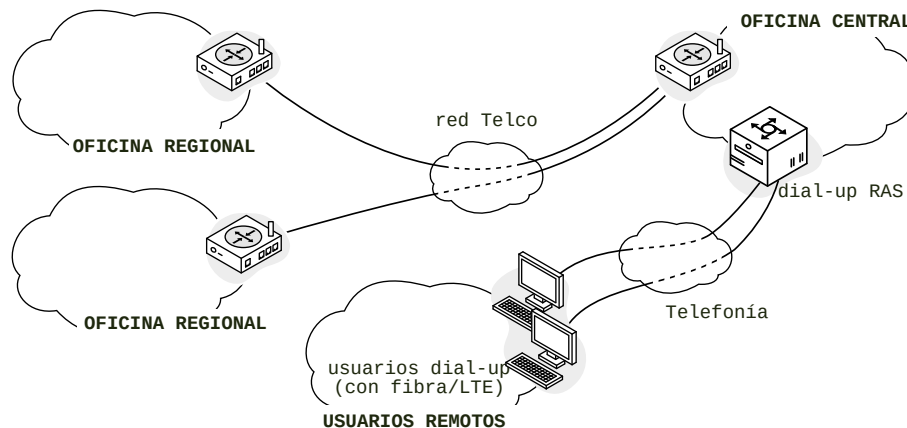


FIGURA 11.1: Red privada que utiliza líneas alquiladas

Partiendo de una topología de este tipo, si alguna de las oficinas tiene acceso a Internet, puede ofrecer conectividad a algunas o todas las demás oficinas. Este esquema (que se muestra en la Figura 11.2) es lo que se denomina *red híbrida*.

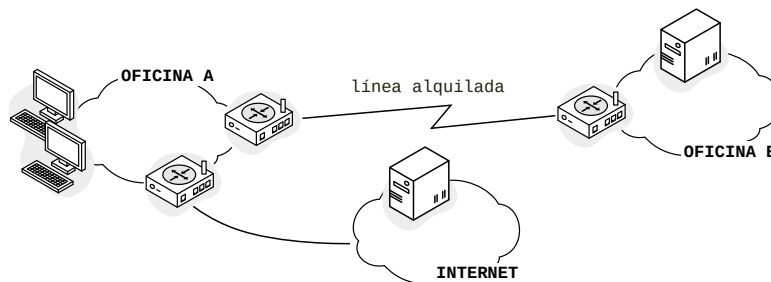


FIGURA 11.2: Red privada híbrida

²Abreviado habitualmente «telco».

11.2. Redes privadas TCP/IP

Precisamente por ser privada, la organización tiene plena libertad para elegir cualquier pila de protocolos disponible, o incluso implementar una tecnología propia.

En la práctica, parece buena idea utilizar la pila TCP/IP por variedad y disponibilidad de software, hardware de red, soporte y sobre todo, como forma de simplificar la conexión de la red privada con otras redes. La única razón para no utilizar TCP/IP en la red privada es que no cumpla con algún requisito técnico específico.



Durante los primeros 90, Novell NetWare IPX tuvo gran popularidad como protocolo para redes privadas, debido a varios factores: tarjetas NIC asequibles, soporte por defecto en Microsoft Windows y en los primeros videojuegos multijugador en red (local) como *p. ej.* Quake, etc. Pocos años después, con la llegada de Internet, IPX fue desbancado rápidamente por IP.

Una *intranet* es justamente una red privada que utiliza tecnología TCP/IP. Sin embargo, el nombre *intranet* se utiliza hoy día, erróneamente, para identificar una o varias aplicaciones o servicios (normalmente web) destinados específicamente al personal de una organización para lo que requiere autenticación específica.

Como caso particular de *intranet*, la *extranet* permite a ciertos usuarios o servicios acceder a los recursos de la red privada desde el exterior. Este acceso está controlado mediante algún sistema de autenticación y autorización.

11.3. Direccionamiento privado

Si la red privada está aislada, la organización tiene libertad para decidir cómo plantear el direccionamiento de sus dispositivos. El diseñador de la red privada tiene tres alternativas:

- Solicitar un bloque de direcciones públicas globales. Implica realizar una petición, y su correspondiente pago, a las autoridades de Internet: IANA o las entidades regionales (RIR) en las que haya delegado la tarea de asignación de direcciones. Si la red privada está aislada, o al menos sus computadores no van a proporcionar servicios hacia Internet, puede ser un gasto injustificado.

- Utilizar un bloque público arbitrario sin conocimiento de las autoridades. Si efectivamente la red privada va a estar esencialmente aislada, no supone ningún problema técnico, pero puede plantear graves problemas logísticos y administrativos si en el futuro esa red acaba siendo conectada a Internet.
- Utilizar uno de los bloques reservados específicamente para redes privadas.

Esta tercera es la opción recomendada por las autoridades en la RFC 1918 [23] y consiste en la elección arbitraria de uno de los bloques definidos en el Cuadro 11.1. A estas direcciones se las denomina simplemente «direcciones privadas».

inicio		fin	prefijo CIDR
10.0.0.0	-	10.255.255.255	10/8
172.16.0.0	-	172.31.255.255	172.16/12
192.168.0.0	-	192.168.255.255	192.168/16

CUADRO 11.1: Bloques IP reservados para direccionamiento privado

Las direcciones privadas deben ser consideradas *no routables* en Internet, es decir, los routers del ISP y de la WAN en general descartarán cualquier paquete IP que tenga como destino una dirección privada. Precisamente por esto, cualquier organización puede elegir uno de estos bloques privados sin necesidad de autorización. Aunque existan millones de redes alrededor del mundo que estén utilizando exactamente el mismo bloque no hay problema, ya que su tráfico nunca podrá ser confundido con el de otra red. Es decir, las direcciones privadas deben ser localmente únicas, pero al contrario de las públicas, no son globalmente únicas.

Aunque el núcleo de Internet (esencialmente BGP) y los ISP descarten este tráfico, la organización puede encaminarlos dentro de su interred corporativa. Esto le otorga gran flexibilidad a la organización, pudiendo crear varias subredes interconectadas para diferentes propósitos o comunidades de usuarios.

11.4. Direccionamiento de enlace local

Las direcciones privadas permiten construir una interred corporativa completa, con múltiples subredes interconectadas. Existe un ámbito todavía más restringido: el de las «direcciones de enlace local» (*link-local addresses*), que solo sirven para comunicar directamente a dispositivos conectados a un mismo enlace. A diferencia de las direcciones privadas, un router nunca

debe encaminar un paquete con origen o destino en este rango, ni siquiera hacia otra subred de la propia organización: quedan necesariamente confinadas al enlace en el que se generan.

En IPv4 el bloque reservado para este fin es **169.254.0.0/16** [24]. Cuando un nodo no logra contactar con ningún servidor DHCP, se puede autoasignar aleatoriamente una dirección de este bloque, tras comprobar con ARP que ningún otro vecino la está utilizando. Es un mecanismo de último recurso que permite que los equipos de una misma LAN se puedan comunicar aunque falle la infraestructura de configuración automática.

En Debian/GNU-Linux esta autoasignación la hace el servicio **avahi-autoipd**. Basta con instalar el paquete para activarlo. Si una petición DHCP falla, expira o agota el tiempo de espera, **avahi-autoipd** arranca automáticamente y asigna la dirección; si más tarde **dhclient** consigue una concesión real, el mismo mecanismo lo detiene.

Se puede probar con una interfaz virtual ejecutando los siguiente:

```
# modprobe dummy
# ip link add dummy0 type dummy
# ip link set dummy0 arp on
# ip link set dummy0 up
```

Esta interfaz **dummy0** no está conectada con nada, no tiene acceso a un servidor DHCP y por tanto, no tiene dirección IP.

```
~$ ip addr show dev dummy0
4: dummy0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default
    ↪ qlen 1000
    link/ether 5e:38:c9:81:30:e1 brd ff:ff:ff:ff:ff:ff
```

Al arrancar **avahi-autoipd** se asigna una dirección de enlace local:

```
~# avahi-autoipd --debug dummy0
[...]
Successfully claimed IP address 169.254.12.100
[...]
```

Una vez conseguido, la interfaz tiene una dirección IP de enlace local:

```
~$ ip addr show dev dummy0
4: dummy0: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default
    ↪ qlen 1000
    link/ether 5e:38:c9:81:30:e1 brd ff:ff:ff:ff:ff:ff
    inet 169.254.12.100/16 brd 169.254.255.255 scope link dummy0:avahi
        valid_lft forever preferred_lft forever
```

La etiqueta *scope link* indica precisamente que la dirección solo tiene alcance en el enlace local.

Todo este mecanismo de autoconfiguración tiene se denomina IPv4LL, según la propia RFC 3927 [24].

11.5. Conectividad en redes privadas

Las redes domésticas actuales, y la mayoría de las que se utilizan en empresas y organizaciones pequeñas y medianas, técnicamente son *redes privadas híbridas con tecnología TCP/IP*. En una red doméstica, el ISP proporciona un dispositivo (ver Figura 11.3), que llamamos informalmente como *router doméstico*. Este router es el que proporciona la conexión de tus dispositivos a Internet. En realidad ese dispositivo (esa caja) no es solo que un router. Incorpora normalmente varios dispositivos y funciones:

- Un router IP con 2 interfaces: LAN y WAN.
- Un conmutador Ethernet.
- Un punto de acceso WiFi.
- Un módem que puede utilizar distintas tecnologías: ADSL, SDSL o actualmente fibra óptica con FTTH, HFC y otras. En las conexiones de fibra, el módem puede ser un dispositivo aparte llamado ONT o estar integrado también en la misma caja.
- Un pequeño computador habitualmente con alguna variante de Linux empujado.
- Un servidor DHCP.

En la Figura 11.3 se pueden ver las conexiones del router doméstico. El primer conector de la derecha (WAN) conecta el equipo con el ONT. Los conectores amarillos numerados de 1 a 4 son los puertos del conmutador Ethernet incorporado. Y la antena, es parte del punto de acceso WiFi.

Cuando conectas tu computador, móvil, televisor, etc. —sea con Ethernet o WiFi— el servidor DHCP incorporado le asigna una dirección privada. El bloque de direcciones configurado suele ser `192.168.0.0/24`, aunque puede ser cualquiera del Cuadro 11.1, y normalmente el propio usuario tiene la posibilidad de cambiarlo accediendo a la web de administración del router.

Si la Internet pública descarta los paquetes IP dirigidos a direcciones privadas (§ 11.3), ¿cómo pueden entonces estos nodos comunicarse con servidores públicos? La solución consiste en *traducir* las direcciones privadas a direcciones públicas al salir de la red privada. Esta traducción la realiza un proceso llamado NAT que se ejecuta en el router doméstico, y por eso se le llama «router NAT».

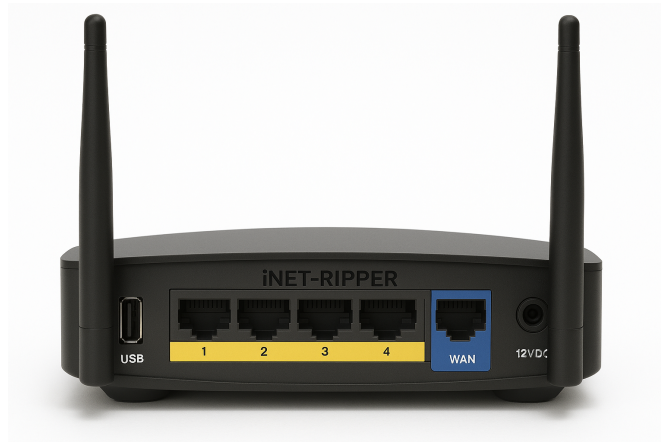


FIGURA 11.3: Conexiones de un router doméstico

11.6. Traducción de Direcciones de Red (NAT)

NAT (Network Address Translation) [25] es un programa (software) que traduce (reescribe) las direcciones de los paquetes IP que reenvía. La topología es similar a la Figura 11.4. El router NAT tiene una interfaz WAN (que conecta con la red del ISP) con una dirección IP pública (*p. ej.* 180.20.30.13) asignada por un servidor DHCP del ISP e interfaces LAN con direcciones privadas (192.168.0.0/24) asignadas por el servidor DHCP del router doméstico.

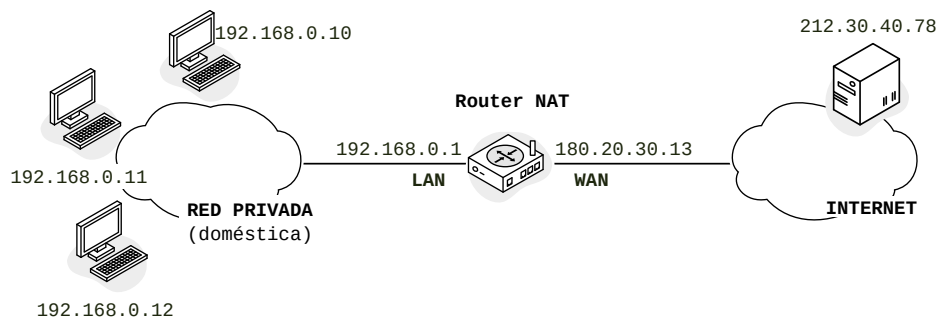


FIGURA 11.4: Ejemplo de configuración NAT en una red doméstica

El uso más habitual de NAT es el que permite a los nodos de la red privada establecer conexiones con servidores de la red pública. Es el llamado SNAT que se muestra en la Figura 11.5. Si por ejemplo, el nodo A quiere conectar con un servidor público, cuando sus paquetes IP salen de la red, el router

substituye la dirección **origen**³ (privada) por la dirección pública (WAN) del router. Al volver la respuesta procedente del servidor remoto, el router substituye la dirección destino (que es la pública del router) de nuevo por la dirección privada del nodo (ver Figura 11.5). Ten en cuenta que este mecanismo es transparente para el servidor y para cualquier otro elemento en Internet, es decir, ni participan ni son conocedores de que esta traducción está ocurriendo.

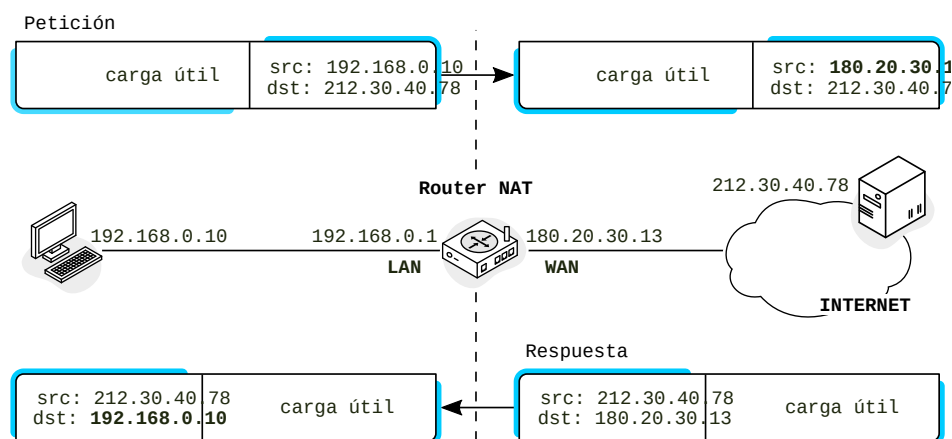


FIGURA 11.5: Ejemplo de SNAT

Para que la traducción de la respuesta funcione, el router debe saber cuál de los computadores de la red privada hizo la petición. Para ello, el software NAT apunta en la tabla NAT⁴ esas correspondencias en el momento de hacer la traducción de salida. Lógicamente, estas entradas en la tabla desaparecen tras un tiempo de inactividad o bien cuando se detecta el cierre de la conexión (si es que el flujo es TCP). El Cuadro 11.2 muestra la tabla tras dos conexiones desde los nodos A y B a dos servidores remotos diferentes.

Dirección local	Dirección remota
192.168.0.10	212.30.40.78
192.168.0.12	200.25.34.56

CUADRO 11.2: Tabla NAT para el envío de la Figura 11.5

Sin embargo, la información de esta tabla es insuficiente. La traducción de direcciones en la respuesta es ambigua si dos o más nodos de la red privada

³Por eso se llama *Source NAT*

⁴Formalmente: *Address Translation Table*

contactan con el mismo servidor remoto a la vez. Es ambigua en el sentido de que el router no tiene forma de saber qué nodo privado fue el que envió la petición que corresponde a esta respuesta.

Para reducir esta ambigüedad existe una técnica mejorada denominada NAPT (Network Address Port Translation) [26]. Consiste en incorporar a la tabla NAT los puertos origen y destino TCP o UDP. La probabilidad de que dos nodos de la red privada elijan el mismo puerto origen en conexiones simultáneas a un mismo servidor remoto es realmente muy baja, y ese sería el único caso en el que se presentaría la ambigüedad. La Figura 11.6 y la Tabla 11.3 ilustran esta técnica. Prácticamente cualquier traducción de direcciones de este tipo es alguna variante de NAPT, pero comúnmente se llama simplemente NAT a todas ellas, y así lo haremos en adelante.

Endpoint local	Endpoint remoto	Proto
192.168.0.10:4045	212.30.40.78:80	TCP
192.168.0.12:32400	200.25.34.56:22	TCP

CUADRO 11.3: Tabla NAPT para el envío de la Figura 11.6

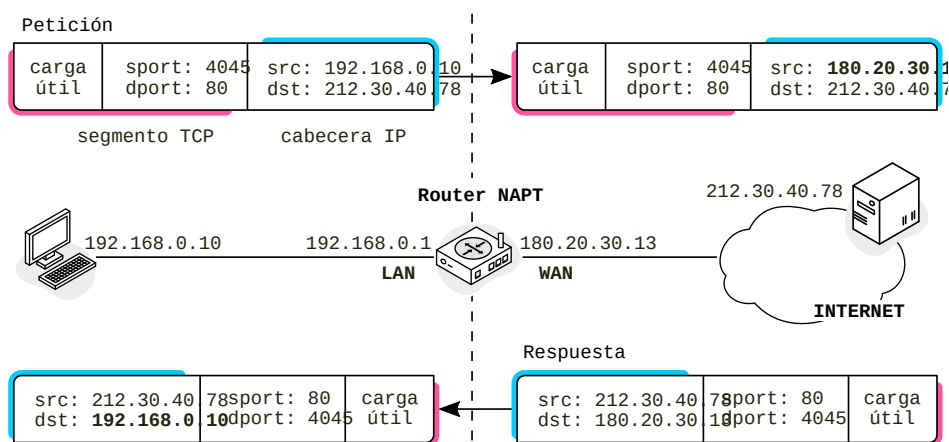


FIGURA 11.6: Ejemplo de NAPT

En todo caso, muchas implementaciones de NAT optan por evitar completamente la ambigüedad traduciendo el puerto del cliente local por un puerto ficticio que el router puede elegir garantizando así que será único en la tabla. Este puerto se conoce como «puerto mapeado» o «sintético». La IP externa del router (normalmente pública) junto con un puerto mapeado forman un *endpoint mapeado*, que es la forma de identificar un flujo desde el exterior de la red privada.

Esta situación se muestra en la Figura 11.7 y el Cuadro 11.4.

Endpoint local	Endpoint mapeado	Endpoint remoto	Proto
192.168.0.10:4045	180.20.30.13:7312	212.30.40.78:80	TCP
192.168.0.12:2400	180.20.30.13:45012	200.25.34.56:22	TCP

CUADRO 11.4: Tabla NAPT con puertos mapeados para el envío de la Figura 11.6

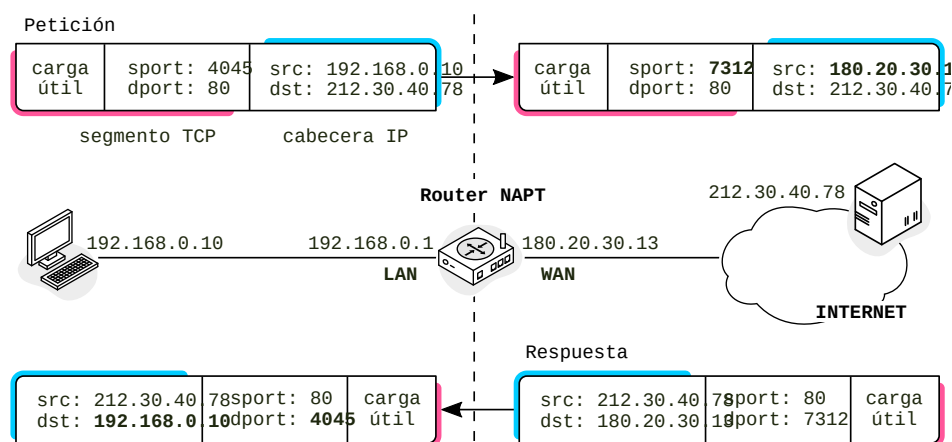


FIGURA 11.7: Ejemplo de NAPT con puertos mapeados

Las entradas de la tabla NAT no son permanentes; si lo fueran, la tabla crecería sin límite y el router acabaría por agotar los puertos mapeados disponibles. Por eso cada entrada tiene asociado un tiempo de inactividad (*idle timeout*): es un contador que se reinicia con cada paquete del flujo y que, al expirar, provoca la eliminación de la entrada.

El tiempo de inactividad depende del protocolo. Con TCP, el router puede detectar cuando se cierra la conexión analizando las cabeceras, y eliminar la entrada de la tabla en ese momento. Por eso, puede otorgar tiempos de inactividad amplios: al menos dos horas según la RFC 5382 [27], pero en casos reales puedes ser días. UDP, en cambio, no tiene conexión ni cierre, así que el router no puede saber si el flujo ha terminado. La estrategia en ese caso es fijar tiempos cortos, entre 30 segundos y pocos minutos según la RFC 4787 [28].

Esto tiene una consecuencia negativa para estos flujos UDP. Si la frecuencia de los mensajes es menor a la que implica ese tiempo, la entrada se borra y el flujo se interrumpe. Para evitarlo, la aplicación debe enviar periódicamente

mensajes de mantenimiento (*keep alive*) para así reiniciar el contador y que la entrada siga activa.

11.6.1. Averiguar la dirección pública

Una consecuencia directa de NAT es que un nodo de la red privada no conoce con qué dirección IP se le ve desde Internet. El comando `ip addr` (Figura 10.1) solo muestra su dirección privada; la pública pertenece a la interfaz WAN del router, y la forma de averiguarla sin la intervención de terceros es consultar el propio router, quizá a través de su web de administración, algo que es distinto para cada fabricante.

Cuando eso no es posible o no resulta adecuado, la alternativa es consultar un servidor externo. Cualquier servidor de Internet con el que se comunique el nodo del usuario recibe necesariamente los paquetes con la dirección traducida, de modo que basta un pequeño servicio que responda con la dirección origen de la petición que le llega. Hay dos medios habituales para lograr esto:

- Servicios web del tipo «cuál es mi IP». Muchos devuelven la dirección en texto plano cuando la petición no procede de un navegador, lo que permite consultarlos desde el terminal con un cliente HTTP como `curl`:

```
~$ curl checkip.amazonaws.com
203.0.113.2
```

Estos servicios no están estandarizados y pueden variar mucho en su funcionamiento y disponibilidad. Para ilustrar lo sencillo que es implementar uno de estos servicios mira el ejemplo [🔗/whats-my-ip/whats-my-ip.py](#).

- Servidores STUN, un protocolo diseñado específicamente para este fin, que además de la dirección devuelve el puerto mapeado. Se describe en § 11.10.

11.6.2. Reenvío de puertos

Otro inconveniente de NAT, que NAPT tampoco resuelve, es que todo el mecanismo se basa en que el router aprende del tráfico saliente de la red privada, y usa esa información para realizar la traducción inversa. Eso implica que no es posible poner servidores en los nodos privados que puedan ser alcanzados desde clientes fuera de la red privada. Para resolver esto, el router proporciona un medio para incluir filas permanentes en la tabla NAT.

Estas entradas en la tabla se conocen como «reenvío de puertos» (*port forwarding*). Ten en cuenta que en este caso, el router traduce la dirección IP destino, en lugar de la origen, y por eso se denomina *Destination NAT* (DNAT). Como esta configuración está pensada para dar acceso a un servidor interno, lo habitual es que el usuario que crea una entrada DNAT en la tabla indique tanto el puerto destino del router como el puerto destino del nodo privado, que es donde realmente hay un servidor vinculado.

El caso más evidente es el de un servidor convencional —un servidor web o SSH en casa, un NAS, una cámara IP—, pero hay muchas aplicaciones cotidianas que también necesitan recibir flujos entrantes, aunque el usuario no las perciba como «servidores»:

- Las aplicaciones P2P (*peer-to-peer*), como BitTorrent, en las que cada nodo es cliente y servidor al mismo tiempo. Un nodo que no admite conexiones entrantes solo puede intercambiar datos con los que sí las admiten, lo que reduce mucho su rendimiento; y si todos estuvieran en esa situación, la red no funcionaría en absoluto.
- Las videollamadas, en las que lo deseable es que el audio y el vídeo viajen directamente entre los participantes, sin pasar por un servidor intermedio que añada retardo. Ese flujo directo es, por definición, entrante para al menos uno de los dos extremos.
- Los videojuegos en línea en los que uno de los jugadores hace de anfitrión (*host*) de la partida, o en los que las consolas intercambian directamente el estado del juego. Algunos fabricantes de consolas clasifican el «tipo de NAT » del usuario en función de si es posible o no recibir esos flujos.

En todos estos casos el reenvío de puertos es la solución más directa, pero requiere que el usuario pueda y sepa configurar su router. Cuando no es así, o cuando hay un CGNAT (§ 11.8), las aplicaciones tienen que recurrir a las técnicas de § 11.10.



Muchos routers domésticos ofrecen UPnP, que permite a las aplicaciones que se ejecutan en nodos de la red privada pedir al router la creación de reenvíos de puertos automáticamente, sin intervención del usuario. Y justo por eso se considera un riesgo de seguridad.

11.6.3. Direcciones en la carga útil

El router NAT reescribe únicamente las cabeceras de red y transporte, pero algunos protocolos de aplicación incluyen direcciones IP y puertos del nodo en la carga útil como medio para que el receptor pueda contactar después a través de flujos entrantes nuevos. Cuando eso ocurre tras un NAT, el nodo privado envía su endpoint local, que resulta inútil para la aplicación remota. Estos son dos ejemplos clásicos:

- FTP [29] utiliza una conexión de control y otra distinta para cada transferencia de datos. En el modo «activo», el cliente indica al servidor, con el comando `PORT`, la dirección y el puerto en los que espera la conexión de datos, y es el servidor quien la inicia. Si el cliente está en una red privada, el servidor recibe una dirección como `192.168.0.10`, inalcanzable desde Internet, y la transferencia falla.
- SIP [30], el protocolo de señalización de la telefonía IP, negocia los flujos de audio y vídeo mediante descripciones de sesión (SDP [31]) que viajan en el cuerpo de sus mensajes. Cada extremo anuncia en ellas el endpoint en el que quiere recibir los paquetes RTP del otro. Con una dirección privada, la llamada se establece, pero el audio nunca llega.

Hay dos formas de abordar este problema. La primera, en el propio router, con una ALG (Application Layer Gateway) [26]: un módulo que reconoce el protocolo de aplicación, localiza las direcciones dentro de la carga útil, las reescribe con el endpoint mapeado y crea por adelantado la entrada en la tabla NAT para el flujo que va a llegar; en la práctica, un reenvío de puertos automático e implícito. Es transparente para la aplicación, pero exige un módulo específico para cada protocolo, con todas sus particularidades, y resulta inútil si el payload viaja cifrado. Por eso los routers suelen implementar solo los de unos pocos protocolos muy extendidos, como FTP o SIP.



Los ALG de SIP de muchos routers domésticos causan más problemas de los que resuelven —modifican mensajes que no deberían o dejan a medias la reescritura—, hasta el punto de que los proveedores de telefonía IP suelen recomendar desactivarlos.

La segunda forma, más robusta, es que la propia aplicación evite anunciar direcciones privadas: bien porque no las necesita —el modo «pasivo» de FTP, en el que también la conexión de datos la inicia el cliente—, bien porque averigua previamente su endpoint mapeado por los medios de § 11.6.1

y es ese el que escribe en el payload. Es la estrategia que recomienda la RFC 3235 [32] y la que siguen las aplicaciones de videollamada actuales (§ 11.10).

11.7. Router NAT con Linux

Cualquier nodo con Linux⁵ y dos interfaces de red también puede ser un router NAPT. Sirva como escenario la misma topología de la Figura 11.4: la interfaz `eth0` (WAN) tiene la dirección pública `180.20.30.13` asignada por el ISP, y la interfaz `eth1` (LAN) tiene la dirección privada estática `192.168.0.1`, que es la puerta de enlace de los nodos de la red privada `192.168.0.0/24`.

Lo primero es activar el reenvío de paquetes (§ 8.1) para que el nodo se comporte como un router:

```
~# echo 1 > /proc/sys/net/ipv4/ip_forward
```

La traducción de direcciones es tarea de *netfilter*, el sistema de manipulación y filtrado de paquetes del núcleo Linux. El sistema recomendado actualmente para configurarlo es *nftables*⁶, por medio del programa *nft* (■ *nftables*). La configuración se organiza en «tablas» (*tables*), cada una asociada a una familia de protocolos (*ip* para IPv4, *ip6* para IPv6 o *inet* para ambas). Una tabla es una colección de «cadenas» (*chains*) y otros artefactos, como conjuntos o mapas. Una cadena es un conjunto de reglas que se aplican a los paquetes que la atraviesan. Para el SNAT hace falta una tabla y una cadena con estos parámetros:

```
~# nft add table ip nat
~# nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; }'
```

type nat

Es una cadena de traducción de direcciones. Solo la atraviesa el primer paquete de cada flujo; los siguientes se traducen automáticamente según el contenido de la tabla NAT.

hook postrouting

Se aplica a los paquetes que salen del nodo, una vez tomada la decisión de enrutamiento.

priority srcnat

Orden de evaluación estándar para las cadenas SNAT.

La cadena solo necesita una regla:

```
~# nft add rule ip nat postrouting oif eth0 masquerade
```

⁵Eso incluye también a un móvil Android.

⁶Evolución del veterano *iptables*.

Se aplica a los paquetes que salen por la interfaz WAN (`oif eth0`). La acción `masquerade` es una variante de SNAT que utiliza como dirección origen la que tenga la interfaz de salida en cada momento, lo que resulta muy conveniente si la dirección pública es dinámica: aunque el ISP asigne otra, la regla sigue funcionando. Con una dirección fija sería `snat to 180.20.30.13`.

Con esto el router está completo: los nodos de la red privada ya pueden establecer conexiones con servidores de Internet. La tabla NAT la mantiene el subsistema de seguimiento de conexiones del núcleo (*connection tracking*) y se puede consultar con el programa `conntrack`:

```
~# conntrack -L
tcp 6 431998 ESTABLISHED src=192.168.0.10 dst=212.30.40.78 sport=4045 dport=80
    src=212.30.40.78 dst=180.20.30.13 sport=80 dport=7312 [ASSURED] use=1
```

Cada entrada contiene dos tuplas. La primera describe los paquetes salientes tal como los envía el nodo privado; la segunda, los paquetes de respuesta que el router espera recibir: dirigidos a su dirección pública y al puerto mapeado 7312. Es decir, esta entrada equivale a la primera fila del Cuadro 11.4. El número 431998 es el tiempo de inactividad de la entrada (§11.6). En Linux es de 5 días para las conexiones TCP establecidas. Para UDP el plazo es de 30 segundos para tráfico en un sentido, y 120 para tráfico en ambos sentidos.

Los ALG de §11.6.3 en Linux se denominan *helpers*: módulos del núcleo como `nf_conntrack_ftp` o `nf_conntrack_sip` que inspeccionan la carga útil de los flujos que tienen asignados y crean las entradas adicionales que necesitan. Con `nft`, se puede usar la sentencia `ct helper` para asignar un *helper* a un flujo.

El reenvío de puertos requiere una segunda cadena, asociada al *hook prerouting*, que procesa los paquetes recibidos **antes** de la decisión de encaminamiento. Tiene que ser así porque la dirección destino debe reescribirse antes de decidir hacia dónde debe ir el paquete.

```
~# nft add chain ip nat prerouting '{ type nat hook prerouting priority dstnat; }'
~# nft add rule ip nat prerouting iif eth0 tcp dport 8080 dnat to 192.168.0.10:80
```

Esta regla DNAT crea la entrada permanente en la tabla: las conexiones que lleguen por la interfaz WAN al puerto 8080 se entregan al puerto 80 del nodo `192.168.0.10`, donde hay un servidor web. Como corresponde a una entrada de reenvío de puertos, la regla indica tanto el puerto destino en el router como el puerto real del servidor interno.

Configuración completa de la tabla:

```
~# nft list table ip nat
table ip nat {
  chain postrouting {
    type nat hook postrouting priority srcnat; policy accept;
    oif "eth0" masquerade
  }
  chain prerouting {
    type nat hook prerouting priority dstnat; policy accept;
    iif "eth0" tcp dport 8080 dnat to 192.168.0.10:80
  }
}
```

Como el resto de la configuración de red, estas reglas también se pierden al reiniciar el equipo. En Debian, el servicio `nftables` carga en el arranque las reglas guardadas en el archivo `/etc/nftables.conf`. Con el siguiente comando se puede guardar la configuración actual para cargar en el siguiente reinicio del nodo o el servicio —lo que hace el segundo comando.

```
~# nft list ruleset > /etc/nftables.conf
~# systemctl enable nftables.service
```

El parámetro `net.ipv4.ip_forward` se hace persistente en el archivo de configuración `/etc/sysctl.conf` (§ 2.10).

11.8. CGNAT (Carrier-Grade NAT)

El direccionamiento privado y NAT sirvieron para aliviar la escasez de direcciones IPv4 durante años. Con estos mecanismos cada hogar solo necesita una dirección pública, aunque tenga muchos dispositivos conectados. Pero con el crecimiento de Internet incluso eso es insuficiente. IANA repartió sus últimos bloques libres en 2011, y los registros regionales fueron agotando después sus reservas, el último fue RIPE, en 2019.

La solución que aplican los ISP consiste en aplicar doble NAT: un router CGN o CGNAT (Carrier-Grade NAT) [33] es un router NAT de gran capacidad situado en la red del ISP, que permite compartir cada dirección pública entre cientos de clientes. Con CGNAT, la interfaz WAN del router doméstico del abonado no recibe una dirección pública, sino una del bloque `100.64.0.0/10` [34], reservado específicamente para este propósito. El tráfico hacia Internet sufre entonces *dos* traducciones consecutivas —en el router doméstico y en el router CGNAT—, esquema que se conoce como NAT444 por las tres direcciones IPv4 que tiene cada paquete durante su tránsito.

En redes de telefonía móvil la práctica está aún más extendida: el número de abonados por estación base es tan alto que las conexiones de datos por

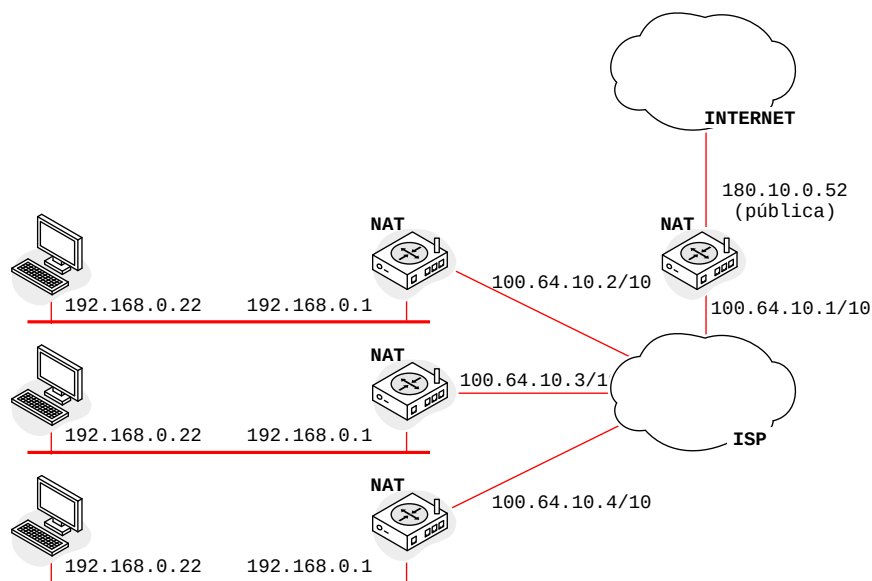


FIGURA 11.8: Configuración típica de CGNAT

LTE o 5G suelen estar tras CGNAT incluso cuando la línea fija del mismo operador no lo está.

Para el usuario final, CGNAT tiene todos los inconvenientes de NAT, pero por partida doble y, además, fuera de su control:

- El reenvío de puertos (§ 11.6.2) no sirve en este caso. Montar un servidor en la red privada accesible desde Internet ya no es posible.
- La dirección pública se comparte con cientos de otros abonados del ISP. Si un proveedor de servicio bloquea esa dirección por el abuso de uno de ellos, el bloqueo afecta a todos.
- La dirección pública ya no identifica a un cliente, por lo que el ISP está obligado a registrar también los puertos mapeados para poder llevar un registro de conexiones⁷.

Con CGNAT, la web de administración del router doméstico ya no sirve para conocer la dirección pública, porque su interfaz WAN no la tiene. Los servicios externos vistos en § 11.6.1 sí la devuelven, ya que ven la dirección origen de los paquetes, es decir, la del router CGNAT tras las dos traducciones. Eso proporciona también una forma sencilla de comprobar si la conexión está detrás de un CGNAT: basta comparar ambas direcciones.

⁷Este registro es obligatorio por ley en muchos países.

Si no coinciden, hay una traducción adicional por el camino; y si además la WAN pertenece a `100.64.0.0/10`, es un CGNAT con seguridad. También suele delatarlo el primer salto después del router doméstico:

```
~$ traceroute -n 8.8.8.8
1 192.168.0.1 1.104 ms 1.087 ms 1.075 ms
2 100.64.0.1 9.371 ms 9.355 ms 9.342 ms
3 180.20.30.1 10.240 ms 10.227 ms 10.213 ms
...
```

Muchos ISP ofrecen la posibilidad de obtener una dirección pública exclusiva⁸, aunque puede conllevar un coste.

11.9. Tipos de NAT

El mecanismo NAT explicado en §11.6 se denomina «simétrico» y se caracteriza por utilizar un puerto mapeado diferente para cada flujo saliente —identificado por endpoint remoto. No es la única forma de implementar NAT.

Distintas implementaciones pueden aplicar restricciones diferentes que pueden afectar a su comportamiento y esto da lugar a diferentes tipos. La taxonomía *clásica* [35] utiliza el concepto de «cono» como metáfora de la correspondencia entre el endpoint mapeado (el vértice del cono) y los endpoints remotos que tienen permitido utilizarlo (la base del cono).

Los tipos son los siguientes (ordenados desde el más permisivo al más restrictivo). Se utilizan los datos de la Figura 11.7 para los ejemplos:

Full cone

El endpoint mapeado acepta mensajes de cualquier endpoint externo: mensajes desde cualquier IP y puerto.

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	*:*	respuesta aceptada

Restricted cone

El endpoint mapeado solo acepta mensajes de endpoints externo que provengan de una IP contactada.

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:*	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	1.2.3.4:1200	respuesta rechazada

Port restricted cone

Solo acepta mensajes que provengan del endpoint contactado exacto.

⁸Coloquialmente se dice «sacarte del CGNAT»

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:80	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:1200	respuesta rechazada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	1.2.3.4:80	respuesta rechazada

Simétrico

Aplica la misma restricción que *port restricted*, pero crea un endpoint mapeado diferente (un puerto diferente) para cada flujo saliente.

Algunas aclaraciones necesarias:

- Todos los tipos requieren que los mensajes entrantes utilicen el mismo protocolo (UDP o TCP) que la petición que creó la entrada en la tabla NAT.
- Para un mismo endpoint local, los tres primeros tipos (cono) utilizan el mismo puerto mapeado aunque el endpoint remoto cambie; hay una correspondencia 1:N, ese es el vértice del cono. En el NAT simétrico la correspondencia es N:N; no hay cono.

Es posible compartir el puerto mapeado para varios flujos porque la tabla NAT puede determinar cuál es el endpoint local a partir del endpoint remoto.

Un ejemplo para *full cone*:

Endpoint local	endpoint mapeado	Endpoint remoto	
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.78:80	petición
192.168.0.10:4045 →	180.20.30.13:7312 →	212.30.40.79:81	petición
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.78:80	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	212.30.40.79:81	respuesta aceptada
192.168.0.10:4045 ←	180.20.30.13:7312 ←	*:*	respuesta aceptada

Estos son los cuatro tipos mas habituales, pero la RFC 4787 [28] descompone el comportamiento de NAT en 9 tipos organizados en dos ejes ortogonales: cómo se asigna el puerto mapeado y restricción de acceso, es decir, qué endpoints remotos tienen permitido utilizarlo:

Asignación de puerto mapeado:

EIM (Endpoint-Independent Mapping) El endpoint mapeado es el mismo para todos los flujos salientes de un mismo endpoint local.

ADM (Address-Dependent Mapping) El endpoint mapeado es específico para cada IP remota.

APDM (Address and Port-Dependent Mapping) El endpoint mapeado es específico para cada endpoint remoto.

Restricción de acceso:

EIF (Endpoint-Independent Filtering) El endpoint mapeado acepta tráfico de cualquier endpoint remoto.

ADF (Address-Dependent Filtering) Solo acepta tráfico de IPs remotas contactadas previamente.

APDF (Address and Port-Dependent Filtering) Solo acepta tráfico de endpoints remotos contactados previamente.

Lógicamente, los tipos clásicos se corresponden con combinaciones de estos dos ejes: *full cone* es EIM/EIF, *restricted cone* es EIM/ADF, *port restricted cone* es EIM/APDF y *simétrico* es APDM/APDF.

11.10. NAT transversal

En muchas situaciones no es posible configurar el reenvío de puertos, sea porque el usuario no tiene acceso, permiso o conocimientos para ajustar la configuración de su router doméstico o porque se encuentra detrás de un CGNAT. En estos casos se aplican las denominadas técnicas de *NAT traversal* [36]. Todas se apoyan en la misma idea: la entrada en la tabla NAT que crea el tráfico saliente indica qué hacer con el tráfico entrante, que en teoría son respuestas, pero que podemos aprovechar para permitir flujos entrantes.

La idea básica de *NAT traversal* es abrir un flujo saliente, determinar cuál es el endpoint mapeado que el router registra para ese flujo, y comunicárselo al cliente remoto para que pueda enviar sus mensajes a ese endpoint.

11.10.1. STUN

La primera tarea es averiguar el endpoint mapeado asociado a un nuevo flujo. Para eso existen los servidores y el protocolo STUN [37]. Es el segundo de los mecanismos de §11.6.1, y el único que devuelve también el puerto. Se trata de un protocolo muy sencillo encapsulado sobre UDP: el cliente local envía una solicitud **Binding Request** a un servidor STUN público, y este le responde con un mensaje **Binding Response** que contiene justamente el endpoint deseado. Suponiendo que el endpoint fuera **180.20.30.13:7312**, la petición y la respuesta de la sesión STUN serían algo como:

```
Message Type: Binding Response (0x0101)
Magic Cookie: 0x37A3FACD
Transaction ID: 0x5EE3564D15A18B6AB42E0489

Message Type: Binding Response (0x0101)
Transaction ID: 0x5EE3564D15A18B6AB42E0489
```

```
Attribute: XOR-MAPPED-ADDRESS (0x0020)
Family: IPv4 (0x01)
Port: 7312 XOR 0x37A3
Addr: 180.20.30.13 XOR 0x37A3FACD
```

Tanto la dirección IP como el puerto sufren una codificación trivial con xor para evitar que un ALG (§11.6.3) los reconozca y los reemplace al encontrarlos en el payload. La respuesta puede incluir también el mismo endpoint sin codificar, en el atributo `MAPPED-ADDRESS`, por compatibilidad con clientes antiguos; si ambos coinciden es que ningún ALG lo ha modificado, y eso es lo que indica la línea `No ALG: Mapped == XOR-Mapped` de `turnutils_natdiscovery`, un poco más adelante.

Con el programa `turnutils_stunclient` () se puede efectuar una consulta a un servidor STUN público y ver la respuesta:

```
~$ turnutils_stunclient stunserver2025.stunprotocol.org
RFC 5780 response 1
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:58961
```

El protocolo STUN también se puede utilizar para determinar (al menos parcialmente) el tipo de NAT que aplica el router local. Para lograrlo el cliente realiza distintas peticiones para ver cómo se comporta en los dos ejes de la taxonomía de la RFC 4787 [28]: asignación de puerto mapeado y restricción de acceso. La RFC 5780 [38] describe dos procedimientos para determinar ambos ejes, que resumimos aquí. Supongamos que el servidor STUN es `stun.example.net:3478`.

11.10.2. Determinar el Mapeado

- Test I — El cliente envía una petición `Binding Request` a `stun.example.net:3478` (la dirección primaria del servidor) y recibe el endpoint mapeado en el atributo `XOR-MAPPED-ADDRESS` (la línea `UDP reflexive addr` del comando anterior). Si el endpoint mapeado es igual al endpoint local, no hay NAT: el nodo tiene IP pública. En la misma respuesta recibe el atributo `OTHER-ADDRESS`, un endpoint alternativo del mismo servidor STUN (otra dirección IP y otro puerto). Corresponde a la línea `Other addr` del comando anterior.
- Test II — El cliente envía otra petición, esta vez dirigida a la IP alternativa del servidor, pero manteniendo el puerto primario. Si el endpoint mapeado coincide con el del Test I, el NAT es EIM: asigna siempre el mismo puerto externo, sea cual sea el destino. Si no coincide, sigue.

Test III — Una nueva petición al endpoint alternativo. Si el endpoint mapeado coincide con el obtenido en el Test II, el NAT es de tipo ADM. Si no coincide —el NAT asigna un puerto externo distinto para cada destino—, es de tipo APDM (NAT simétrico).

11.10.3. Determinar el Filtrado

- Test I — El cliente envía una petición **Binding Request** normal a la dirección primaria del servidor, confirmando que hay conectividad UDP básica.
- Test II — El cliente envía otra petición a la dirección primaria del servidor, activando el atributo **CHANGE-REQUEST** para que responda desde el endpoint alternativo. Si llega la respuesta, el NAT es EIF (acepta paquetes entrantes desde cualquier origen). Si no llega respuesta, el NAT está filtrando por origen, y es necesario el Test III para saber si es por IP o puerto.
- Test III — Nueva petición a la dirección primaria con **CHANGE-REQUEST**, pero solicitando únicamente el cambio de puerto (misma IP principal). Si llega la respuesta, el NAT es ADF (filtra por IP remota). Si no llega, el NAT es APDF.

Es importante precisar que el cliente debe utilizar el mismo socket (mismo puerto local) para todas las peticiones, de manera que el router mantenga la misma entrada en la tabla NAT.

El programa `turnutils_natdiscovery` aplica este procedimiento:

```
$ turnutils_natdiscovery -m -f stunserver2025.stunprotocol.org
-- Mapping Behavior Discovery --
=====
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Response origin: : 3.22.142.132:3478
0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:37140
0: : IPv4. Local addr: : 0.0.0.0:37140
=====
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.135.212.85:3478
0: : IPv4. Other addr: : 3.22.142.132:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:37140
0: : IPv4. Local addr: : 0.0.0.0:37140
=====
NAT with Endpoint Independent Mapping!
=====

-- Filtering Behavior Discovery --
=====
No ALG: Mapped == XOR-Mapped
0: : IPv4. Response origin: : 3.22.142.132:3478
```

```

0: : IPv4. Other addr: : 3.135.212.85:3479
0: : IPv4. UDP reflexive addr: 203.0.113.2:60547
0: : IPv4. Local addr: : 0.0.0.0:60547
STUN receive timeout..
STUN receive timeout..
=====
NAT with Address and Port Dependent Filtering!
=====

```

11.10.4. Señalización

Con el procedimiento anterior, ambos extremos deben determinar su endpoint mapeado, si es que ambos están detrás de un NAT. La segunda tarea es comunicar sus endpoints (público o mapeado) a su contraparte. Para intercambiar estos endpoints se utiliza el servidor de señalización o *rendezvous* [36].

El servidor de señalización es un nodo público que ambos extremos conocen y al que pueden conectarse (usualmente con TCP). Piensa que estos extremos podrían ser clientes de un servicio de videollamada, o de un videojuego en línea. La comunicación con el servidor de señalización ocurriría en la fase de espera a otros participantes o jugadores (el «lobby»). El servidor de señalización no interviene en la comunicación entre los extremos, solo sirve para intercambiar sus endpoints mapeados.

No hay un protocolo o especificación normalizada para la señalización porque depende mucho de cada aplicación específica. Debe manejar datos particulares del dominio de la aplicación (creación y registro en salas, partidas, reuniones, etc.) y seguramente también tendrá un sistema de registro y autenticación de usuarios.

El siguiente es un ejemplo simplificado, en el que A es un nodo detrás de NAT, B es un nodo con IP pública y S es un servidor de señalización y el objetivo es que B pueda establecer una conexión activa con un servidor (up programa) en A.

```

A -> S: Quiero unirme a la partida AX23.
A -> S: Mi endpoint mapeado es 180.20.30.13:4500 (descubierto con STUN).
B -> S: Mi endpoint público es 200.10.20.12:3200 (descubierto con STUN).
S -> B: El endpoint de A es 180.20.30.13:4500.
S -> A: El endpoint de B es 200.10.20.12:3200.

```

En realidad, siendo B un nodo público, no sería necesario que A obtenga su endpoint a través del servidor. En una situación en la que los dos extremos están detrás de sendos NAT sería imprescindible el intercambio mutuo de endpoints.

En este punto, si el NAT de A es de tipo *full-cone*/EIM, B puede abrir una conexión TCP directamente con A a través su endpoint mapeado. El router le entregará esos paquetes, porque la tabla NAT así lo indica: acepta cualquier origen remoto.

Sin embargo, si el NAT de A es de tipo *restricted-cone*/ADF o *port-restricted-cone*/APDF, los mensajes de B serán descartados. Es necesario aplicar una técnica adicional: *hole punching*.

11.10.5. *hole punching*

La técnica de *hole punching*⁹ consiste en que una vez conocido el endpoint mapeado de la contraparte, el nodo privado envía un mensaje en el sentido contrario, para que el router registre el origen remoto como permitido. Dicho de otro modo, la «perforación» la realiza el nodo que actuará de servidor en su propio NAT. En el ejemplo anterior, si A crea una conexión al endpoint de B, el NAT de A aceptará mensajes entrantes de B siempre que el NAT sea tipo ADF o APDF.

Esto también funciona si ambos extremos (A y B) están tras un NAT. El siguiente es un listado de mensajes que ilustra la secuencia de eventos para ese caso:

```
A -> S: Quiero unirme a la partida AX23.
B -> S: Quiero unirme a la partida AX23.
A -> S: Mi endpoint mapeado es 180.20.30.13:4500 (descubierto con STUN).
B -> S: Mi endpoint mapeado es 190.33.44.10:6500 (descubierto con STUN).
S -> B: El endpoint de A es 180.20.30.13:4500.
S -> A: El endpoint de B es 190.33.44.10:6500.
A -> B: Perforación. El \NAT de A registra el endpoint de B como permitido, El \NAT de B
↔ lo descarta.
B -> A: Perforación. El \NAT de B registra el endpoint de A como permitido, El \NAT de A
↔ lo descarta.
A -> B: Mensaje aceptado.
B -> A: Mensaje aceptado.
```

El «agujero» no es más que una entrada en la tabla NAT de cada extremo y, como tal, tiene asociado un tiempo de inactividad (§ 11.6). Si el flujo puede pasar periodos en silencio —una partida en pausa, una llamada sin audio—, ambos extremos deben enviar los mensajes *keep alive* con la frecuencia adecuada para reiniciar el contador antes de que expire (cada 15–20 segundos). El contenido del mensaje puede ser ignorado por el receptor. También se puede enviar una petición STUN, que además permite comprobar que el endpoint mapeado no ha cambiado.

⁹en español: «perforación de agujeros», traducción que por suerte nadie usa.

Sin embargo, ni siquiera con esto se puede atravesar un NAT simétrico (o ADM/ADF) porque el endpoint mapeado descubierto con STUN no sirve para la conexión inversa. Cuando **A** trate de crear un flujo hacia **B**, el NAT de **A** asignará un puerto mapeado distinto y el NAT de **B** filtrará esos paquetes.

La solución es utilizar un servidor intermedio para la transferencia de datos. Se denominan servidores *relay* o TURN.

11.10.6. TURN e ICE

El servidor TURN [39] es un nodo público que actúa de proxy. Esencialmente ambos extremos establecen una conexión con el servidor y le dan los datos que quieren enviar a su contraparte.

TURN es el último recurso, porque añade latencia y consume ancho de banda en el servidor; efectos muy poco deseables en aplicaciones de video-llamadas o videjuegos, que son las principales candidatas a utilizar *NAT traversal*.

Es sencillo entender que todas estas técnicas de *NAT traversal* implican mayor coste y menor rendimiento cuanto más restrictivo es el NAT. Para simplificar y sistematizar la elección de la técnica más adecuada se utiliza el protocolo ICE [40]. ICE permite a los extremos descubrir sus endpoints mapeados, intercambiarlos y evaluar cuál es la mejor técnica para efectuar la comunicación.

ICE no evalúa la situación. En su lugar, genera una lista de endpoints candidatos y los evalúa por orden de prioridad. La lista incluye (en este orden):

- host**
endpoints locales (porque los extremos podrían estar en la misma red)
- srflx**
endpoints mapeados descubiertos con STUN.
- relay**
endpoints de servidores TURN.

La lista se intercambia a través del servidor de señalización y cada extremo va probando los candidatos hasta que encuentra uno que funciona.

Tanto WebRTC como SIP (dos protocolos muy utilizados en aplicaciones de videollamadas) emplean ICE para establecer la comunicación con NAT transversal.

Y ¿qué más?

Este capítulo ha introducido el concepto de red privada —algo tan cotidiano como desconocido—, sus variantes —líneas alquiladas, redes híbridas, intranets y extranets— y el papel que juega NAT como su variante CGNAT para paliar el agotamiento de direcciones IPv4. También tiene un alto precio: NAT rompe la conectividad extremo a extremo, perjudica a los protocolos que transportan direcciones en su carga útil e impide la simetría —los nodos de la red privada puede iniciar flujos hacia nodos de Internet, pero no al revés—, de modo que cualquier aplicación que necesite recibir flujos entrantes —videollamadas, videojuegos en red, P2P— debe recurrir a técnicas de *NAT traversal*, que también se han abordado.

Sin embargo, estos problemas son propios de IPv4 y su alarmante escasez de direcciones, que no permite asignar una IP pública a cada dispositivo, como sería deseable —al menos desde la lógica de la arquitectura de Internet.

Con IPv6 el, espacio de direcciones crece de forma exponencial: literalmente miles de millones de direcciones públicas por hogar. Veremos esa y otras ventajas de IPv6 más adelante.